WayFinder AR Indoor Navigation R&D Report - Android Approach and Integration Guide

(Prepared on · October 2026)

Purpose

This document explains how indoor navigation works in the Android prototype, why we chose an ARCore-based visual approach with printed AR signs (the same idea as the iOS prototype), and why we are not using BLE beacons, Wi-Fi RTT or UWB for the drawn AR guidance. It also lists the rules the mapping team must follow so that navigation works reliably for visitors, and it records what we found about accuracy.

The core idea is the same as on iOS: help visitors find their way to destinations inside buildings in Qatar (museums, malls, exhibition halls, airports, hotels and similar venues) using augmented reality on their phone. Android has no ARKit world map, so the Android prototype finds its position from a printed sign and follows a path that an employee walked beforehand.

R&D journey and struggles

We did not repeat the iOS team's evaluation of third-party SDKs (Navigine, ARway Kit, Immersal). We built natively with ARCore (1.57.0), SceneView (4.60.0, Filament rendering), Kotlin and Jetpack Compose. The work went in four steps:

1. Read the Visitor App requirements (Module 1, AR navigation and wayfinding) and planned the approach: ARCore motion tracking plus printed signs that tell the phone where it is. BLE and Wi-Fi RTT were kept as possible later additions, because they need hardware installed in every venue.
2. Ported the iOS WayFinder SDK design to Android one-to-one: the venue model, A* routing, step-free routing, turn-by-turn maneuvers, marker alignment maths and the demo venue package (the same venue.json and sign images as iOS). The 26 iOS unit tests were ported with it.
3. Rebuilt the iOS "Record" idea (ARPersistence) without an ARKit world map: the employee walks, the app records the path as floor points, and objects are placed by tapping the screen. Visitors later follow that recorded path.
4. Tested on two real phones (a Samsung Galaxy A54 and a CPH2661, both Android 16) and fixed what the tests showed. The table below lists the main struggles.

Struggle	What we found	What we did
No way to move the AR origin	ARKit can re-centre its world on a sign (setWorldOrigin). ARCore cannot.	The app keeps an alignment transform (venue pose in the session). All route content hangs under one root node posed by it, and the camera is converted with its inverse.
Tracking never started	On Android 12+ the camera pose stayed at (0, 0, 0) and the log was full of IMU timestamp errors, so no route or arrows could appear.	ARCore needs the full IMU rate. Declaring the HIGH_SAMPLING_RATE_SENSORS permission fixed it; tracking then started at once on both phones.

Struggle	What we found	What we did
Alignment shifted with the sign's tilt	A sign leaning back (for example a laptop screen) moved the whole venue sideways: a 15° lean at 1.4 m height is about 0.36 m.	The alignment now uses only the sign's direction (yaw) and its centre. Covered by a unit test.
Noisy first sighting	A scan from several metres, or from a steep angle, gives a poor sign pose.	Nothing is drawn or recorded until three close, head-on sightings from slightly different viewpoints have been averaged (weighted by distance and viewing angle, yaw averaged on the circle).
Drift while walking	ARCore drifts by centimetres over a long walk, so one alignment leaves error at the far end.	The walk and the objects are recorded in ARCore's own stable frame and converted once when saving. Scanning the same sign again at the end corrects the drift along the path.
Wrong floor height	The venue file assumes the sign is exactly 1.4 m up, so arrows and the pin floated above or sank below the real floor.	The real floor height is detected from ARCore planes, recorded with the path and used for the arrows and the destination pin.
Sign size and mounting	ARCore judges the sign's distance from how big it looks. A sign that is not exactly the declared width shifts the result, differently on every phone and distance.	Exact-size printable PDFs with a 10 cm check bar. The sign must be rigid, flat and never moved.
Other signs bent the path	A second sign is placed by its surveyed position in venue.json, which is wrong if it was hung anywhere else.	A path uses only the sign it was recorded from. The path file stores that sign's id.
Taps and distant objects	A tap with no surface found did nothing, and a small mug several metres away was invisible.	The object is placed on the floor along the tap and the user is told. Far objects are drawn larger.
Early "arrived" pop-up	Arrival fired within 2 m in a straight line, and also for a path that passes near its destination.	Arrival now needs 1 m, being at the end of the route, and staying one second.

Problem and goals

Module 1 of the Visitor App requirements (*AR navigation & wayfinding*) asks for camera-based guidance inside venues. GPS does not work reliably indoors, so the phone needs another way to know where it is and which way it is facing. The R&D aimed to prove the core visitor journey. The table shows where the Android prototype stands.

Goal from the requirements	Status	Android notes
Pick a destination and launch AR navigation	Working	Destination list with search and a step-free toggle; the AR screen opens from it. Verified on both phones.
Arrows, labels and distance in the camera view	Working	Red floor arrows with a pulse, a destination pin, name labels, a guidance banner and a mini map. Placed objects (mug, gift, star, balloon, message) are drawn along the path.

Goal from the requirements	Status	Android notes
Real-time recalculation when the user leaves the route	Implemented	Re-plans after 3 s more than 3 m off the route. Not stress-tested.
Contextual tips and arrival confirmation	Working	Tips come from venue.json. The arrival screen was tuned after testing and confirmed to appear at the right moment.
Offline AR map caching	Partly	The venue package is bundled and paths are stored on the phone, so it works without a network. Downloading from a server is not built (no backend yet).
Accessibility routing (step-free)	Implemented	Step-free routing works on the venue graph. Recorded paths do not yet carry accessibility information.
SDK usable by other entities	Partly	The logic is in two library modules (wayfinder-core and wayfinder-ar); they are not published as a packaged SDK yet.
Reuse mapping data across iOS and Android	Open	The venue and sign data is shared with iOS. Android paths are sign-relative JSON, not an ARKit world map, so they do not depend on one platform's map file. Whether iOS can read them has not been tested.

One app, two modes

Unlike iOS, which has two apps (Record and Visitor), the Android prototype is a single app with the two roles in one flow.

Record: "Add destination" in the app. An employee scans a printed sign, walks to the destination, taps to place objects along the way and saves. This is meant only for personnel mapping the venue; the prototype does not yet lock it behind a role.

Visitor: the destination list. A visitor picks a saved destination, scans the same sign the path was recorded from, and follows the red arrows. Visitors do not edit anything.

Workflow of the Android prototype

Employees record a path once; visitors replay it after the app recognises the sign.

EMPLOYEE MODE

1 Scan face the printed sign; lock-in after 3 close sightings	›	**2 Walk and place** path point every 25 cm; tap to place objects	›	**3 Save** name the destination; one JSON file per path	›	**4 Share** copy the file to another phone

same phone, or copy the file to another phone

VISITOR MODE

5 Load paths are read when the app starts	›	**6 Relocalize** scan the SAME sign it was recorded from	›	**7 Choose** pick a destination from the list	›	**8 Navigate** red arrows, banner, mini map; arrive within 1 m

5. **Scan (employee).** The employee taps Add destination and faces the printed sign from 1 to 2.5 m, moving the phone slowly side to side. After three close head-on

sightings the alignment is locked in and recording starts. Until then nothing is recorded.

6. **Walk and place (employee).** While walking, the app drops a point on the floor about every 25 cm, smoothed against hand sway. Picking an object in the bar and tapping the screen places it on the surface under the finger (mug, gift, star, balloon or a text message). Scanning the same sign again on the way corrects drift.
7. **Save (employee).** Save destination here needs the phone held steady for a moment. The destination is where the employee stands; the path ends there. Everything is converted to venue coordinates once, then written to its own file.
8. **Share (employee).** Each path is one JSON file (paths/<name>.json). Today it is copied to another phone with a script over USB. An in-app export and import, and sending it to a backend, are not built.
9. **Load (visitor).** The path files are read when the app starts and the saved destinations appear in the list.
10. **Relocalize (visitor).** The visitor faces the same sign the path was recorded from. Another sign is deliberately ignored, because its surveyed position would not match. This is the most sensitive step for accuracy (highlighted).
11. **Choose (visitor).** The visitor picks a destination. Routes use only that destination's own recorded path.
12. **Navigate (visitor).** Red arrows on the floor follow the walked path, so they go around walls. A banner and a mini map support the visitor, the placed objects appear where they were put, and the route is re-planned if the visitor leaves it. Arrival is confirmed within 1 m at the end of the route.

Venue data and files

Each venue has one venue package, which is bundled in the app today, and one JSON file per recorded path, stored on the phone. A backend and API are not built yet.

File	Contents
DemoMall.venue/venue.json	The venue: levels, walkable network, destinations (POIs), the signs (id, image, printed width, position, direction, mounting) and tips. The same file iOS uses.
DemoMall.venue/markers/<sign>.png	The sign images ARCore recognises. Exact-size printable PDFs (with a 10 cm check bar) are in local_ar_files/printable_signs.
paths/<destination-name>.json	One recorded path: the destination, the walk, the objects and quality data (fields below).

Field in a path file	Meaning
venueId	The venue the coordinates belong to. Paths of different venues never mix.
destination	Name, level, x, z and floorY (real floor height) of where the path ends.
walkedPoints	The path: floor points in venue metres, grouped by run and numbered in order.
objects	Objects placed along the path: kind, position, turn, size and, for a message, its text.
startMarkerId	The sign the path was recorded from. Visitors must scan this one.

Field in a path file	Meaning
alignmentSpreadCm	How well the sign scans agreed while recording (smaller is better; above about 5 means a poor alignment).
driftCorrectedCm	How much drift a second scan of the sign corrected; empty if the sign was not scanned again.

Deleting a destination deletes its file, so its path and objects go with it. A file the app cannot read is skipped and never overwritten. Earlier builds saved one shared file, and before that three files; the current app splits those into per-path files the first time it starts and keeps the old file as a backup.

What decides accuracy

Every position the app shows is measured from the printed sign, so the sign is the weakest link. These effects are calculated from geometry and are meant to show why the rules below matter; we have not yet measured the real error in a venue.

Cause	Size of the effect	Rule or fix
Sign direction (yaw) off by 1°	about 9 cm at 5 m from the sign, 17 cm at 10 m	Use a bigger sign (40 cm would halve it), scan head-on and from a few viewpoints.
Sign leaning back by 15°	about 36 cm if the tilt is not removed	Fixed in the maths. Still mount the sign flat and vertical.
Sign printed 10% too large or small	about 15 cm along the viewing direction at 1.5 m	Print at 100%, check the 10 cm bar with a ruler.
Sign moved between sessions	the same as the move	Fix the sign rigidly. Never use a laptop screen as the sign.
Drift while walking	a few cm per 10 m	Scan the same sign again at the end of the path.
A different sign scanned	up to metres	Only the recorded sign is used.

The app shows a sign agreement figure while recording (for example "sign ±2 cm") and says how many centimetres of drift a second scan corrected. Both are saved in the path file, so a poor recording can be spotted afterwards.

Summary

We now have a working Android proof of concept for AR indoor navigation that uses printed AR signs plus a recorded walked path. It routes along the walked path, around walls, shows red guidance arrows and placed objects in the camera view, re-plans when the user leaves the route, and confirms arrival. A recorded path was copied to a second phone, loaded and routed there after scanning the sign.

The current build is a proof of concept, not production-ready. It is Android-only, English-only and has been tried only in a small area with two phones. The core logic has 78 automated unit tests; the camera, tracking and accuracy behaviour could only be checked by hand on real phones. We have not yet measured the real error in centimetres, and users reported the accuracy of objects and destinations as lower than wanted, so accuracy is the main open item.

Not built yet: multi-floor paths (a path stays on one level), a backend or downloading, in-app export and import, locking the record mode to employees, TalkBack and vibration on arrival, Arabic text, keeping the screen in portrait, and a packaged SDK.

Notes:

- The approach works in a single room or small area. It depends on a sign that is printed at its exact size, fixed rigidly and never moved, and on scanning that same sign each time. We have not tested larger venues, so we ask the BA and testing teams to evaluate it once the prototype is shared.
- A path is recorded relative to one sign. Longer routes will need more than one sign, with each path anchored to the sign nearest to it, and more accurate surveyed sign positions.
- ARCore must be supported and up to date on the phone. The app declares the high-rate sensor permission that tracking needs on Android 12 and later.
- AR navigation needs **something physical in the venue** to know where the phone is. Here that is the printed sign. Beacons could help, but they cannot replace the signs for drawing arrows in the camera view.
- For handing paths to other teams, the plain JSON path files and the shared venue package are easy to exchange, and are a candidate format for the backend. Whether iOS can read the Android path files is still to be tested.